



Django 訂單管理系統：從 0 到 1 的後端開發歷程

這是什麼&&我學到了什麼

這是我第一個大型且完整的個人專案。從零開始學習後端框架，**我自己包辦了前端的 HTML/CSS/JS、後端的邏輯處理以及資料庫設計，五種程式語言的交互都是自己完成並部署維護。**在這個過程中，我不僅學會了**基礎且流行的開發流程（從建立專案、版本管理到部署）**，也學習了**網路協議（如 HTTP）與 Linux 作業系統操作**。更重要的是，我培養了極強的**自學能力與閱讀官方文檔的習慣**，也**初步建立了基礎架構設計的思維**。這個專案算是我個人開發者生涯的重要起點，在這個專案之前我還是只會寫單python檔爬蟲的小菜鳥，在這個專案之後我已可以初步的完成一個完整的系統了。後來我參加 AIS3 Junior（教育部新型態資安暑期課程）的奪冠之旅，也是基於這次打下的技術與觀念基礎，甚至程式本身。

動機與開發起點

在暑假前一陣子，我從母親那接到了一份不錯的委託：幫他工作的地方設計一個訂單管理系統，主要目的是取代目前繁瑣的 Google 表單和 Excel。其實這個項目最吸引我的是可以獲得一定的零用錢，每當我有一個具體目標時，寫程式的動力就會大增，因此我在開發前期獲得了極大的效率提升，甚至很多時候都是熬夜開發。

在多方比較後，我決定使用比較熟悉的 Python 搭配 Django 框架。相較於 Flask，Django 提供了更完整的 ORM、用戶認證與管理介面，很適合應對這種有權限劃分與資料關聯的複雜系統。選定框架後，我一開始看線上文字教學發現很難吸收，於是轉而跟著一系列教學影片一步步實作，並用以前學過的靜態網頁設計知識，先刻出基礎的 HTML 介面與網頁路由。

資料庫設計與架構思維

一開始，我先研究了 SQL 的基本語法，在 Django 內建的 SQLite 中手動建立了商品、用戶和訂單的資料表。但隨著系統越寫越胖，我發現用原生 SQL 進行資料庫操作過於複雜且難以維護。後來我轉而使用 Django 的 Models（ORM 系統），這才體會到框架的強大。透過 ORM，我能重新設計模型之間的關聯，例如訂單與商品的多對一關係，或是透過 ExtendedUser 來擴充內建的帳號系統以區分用戶權限。這不僅提升了開發效率，更讓我摸索

出了高級架構設計的概念。我第一次體會到一個好的後端系統不能只是程式碼能跑就好，資料庫的關聯怎麼綁定、模組之間怎麼分開，這些早期的架構規劃會直接決定專案後期的擴充與維護難度。

功能實作與踩坑

在開發訂單與用戶管理功能時，我原本是使用純 HTML 表單與 HTTP POST 來傳輸資料，但後來發現 Django 有專屬的 Forms 系統，不僅可以直接用模型(Models)還自帶驗證機制。於是，我將前後端全數改寫。但現在回頭看，我其實有點後悔太早全部改成 Django Forms。因為它的寫法完全被框架綁定，每次想新增一些客製化功能就要重新翻找文檔，這讓我的開發時間大幅增加。

不過，這段頻繁卡關的陣痛期反而成了我收穫最多的地方。它極大地逼出了我的自學能力以及深入研究官方文檔的耐心。我發現遇到問題時，與其在網路上盲目搜尋零碎的教學或依賴 AI 的片段解答，不如直接啃官方文件，理解框架的底層邏輯跟最詳細的說明，這是我認為這個專案帶給我最寶貴的軟實力。

而在權限與安全管控上，我除了要求全站登入驗證外，還自定義了攔截器（Mixin），確保管理員專屬的頁面與 API 不會被一般用戶越權存取。為了讓後台更易用，我也實作了動態分頁功能，讓管理者在瀏覽大量資料時能順暢導航。

系統設計與成果展示

我將路由拆成 `base`、`customer`、`op` 三個 app，分別處理登入與入口、門市下單流程、管理端維運功能。登入機制全面採用 Django 內建的 `django.contrib.auth` 系統搭配自訂 `LoginForm`，確保憑證傳輸的安全。資料庫部分，我以 Django 預設的 SQLite 儲存庫為基礎，將 `Product`、`Order`、`ExtendedUser` 設為核心模型，透過 ORM 建立商品、訂單與使用者的完整關聯；其中 `ExtendedUser` 更以 `OneToOneField(User)` 擴充原生帳號，並用 `user_class` 參數區分身份，讓權限分流與功能切分能安全地接入資料庫層，而不僅是靠前端畫面隱藏。

選擇平台:

平台	品名	上市日期	建議售價	備註	輸入備註	數量	操作
ps5	mario	2024-11-11	2900	aaa		0	- +
ps	hahacho	2025-01-01	400	nonote		0	- +
ps5	111	無	無	無		0	- +
sb	sb	無	無	無		0	- +

17.0.0.1:8000/customer/sherky/

在功能上，我做了一個同一套系統，兩種操作體驗的後端流程。一般門市使用者登入後可進入下單頁，系統會以表單組合對應商品清單，提交後自動建立訂單並綁定到該門市帳號。在查詢頁面中，使用者只能看到同體系門市的訂單；為了提升使用者體驗，我利用ai寫了複雜的 JavaScript (`table.js`) 在前端直接萃取與建立表格 DOM 元素的資料字典，實作出能即時切換平台、過濾特定門市、以及按日期升 / 降冪排序的功能，這不僅操作順暢也減少了伺服器的重複查詢。管理員端則以自訂的 `OpRequiredMixin` 搭配 `LoginRequiredMixin` 進行雙重控管，未授權帳號會被直接攔截。管理員可在後台進行使用者管理、商品資訊維護（如發售日、價格）及訂單檢視等完整的 CRUD 工作，並在清單頁面實作分頁機制 (`paginate_by`)，大幅改善大量資料瀏覽的效率。這一切設計讓系統不只達成了「取代 Excel」的最基本目標，更具備了可持續維護的現代後端架構。(詳細設計極度建議看看我的操作影片:https://youtu.be/ff_TkNODYkY?si=F_5v-KfobxT_7gmX)

版本控制與上線部署

開發中途因為不斷需要回檔，我意識到不能再依賴「Ctrl+Z 加複製備份」了。我上網研究了 Git 的基本指令，熟悉後便開始使用版本控制，雖然坦白說，我後來大多是依賴 VSCode 的 GitLens 或 Git Graph 插件來操作，但這確實解決了程式碼遺失的風險。最後的部署階段也充滿挑戰，原本想用的 Heroku 取消了免費方案，最後我轉戰 Render，從準備環境配置、生成 requirements.txt 到真正部署上線，這完整的流程讓我把本地端的代碼變成了實際運作的產品。

心得與反思

現在回頭看，我很後悔當時沒有保持良好的版本管理與開發筆記習慣，導致過了一段時間要寫紀

錄時忘了許多細節。但總體來說，這是一個非常好的學習體驗。這次的經驗不僅讓我確認了自己**以技術實踐創意的特質**，更鍛鍊出我最核心的競爭力：在資源有限、沒有老師指導的環境下，**透過自學與讀官方文檔來克服技術瓶頸**，還有從無到有完成一個完整的系統。另外，這個專案也讓我**真正搞懂了架構思維**，體會到將資料庫關聯、模組分離等前期規劃做好的重要性，而不是單純寫一個巨大的.py檔，這個部分成為了我未來開發重要的基礎。雖然這個系統最後沒有成功讓我媽真正採用，但我確實地從這個專案學到了很多東西。

額外資料

- 操作示範影片:https://youtu.be/ff_TkNODYkY?si=F_5v-KfobxT_7gmX
- 完整開發筆記:<https://hackmd.io/@Kelsier-64/SycObSkuke>
包含更多的技術與開發細節和程式碼

參考資料

我的github倉庫:<https://github.com/Kelsier64/OrderManageWeb>
django官方文檔:<https://www.djangoproject.com/>
教學影片:<https://youtu.be/rHux0gMZ3Eg?si=uR6KXPsaAcKkfFWbp>